
Computer Programming
Summer Interest Group
Session 21 / Environments

Deuteronomy 7:1–4

“When the Lord your God brings you into the land that you are entering to take possession of it, and clears away many nations before you, the Hittites, the Girgashites, the Amorites, the Canaanites, the Perizzites, the Hivites, and the Jebusites, seven nations gmore numerous and mightier than you, hand when the Lord your God gives them over to you, and you defeat them, then you must devote them to complete destruction. You shall make no covenant with them and show no mercy to them. You shall not intermarry with them, giving your daughters to their sons or taking their daughters for your sons, for they would turn away your sons from following me, to serve other gods. Then the anger of the Lord would be kindled against you, and he would destroy you quickly.

Context for Virtual Environments

Deuteronomy 7:1-4 admonished the Israelites to not contaminate their society with ungodly influences.

Similarly, In Python programming, we are interested in creating a unified application free from outside, undesired behavior.

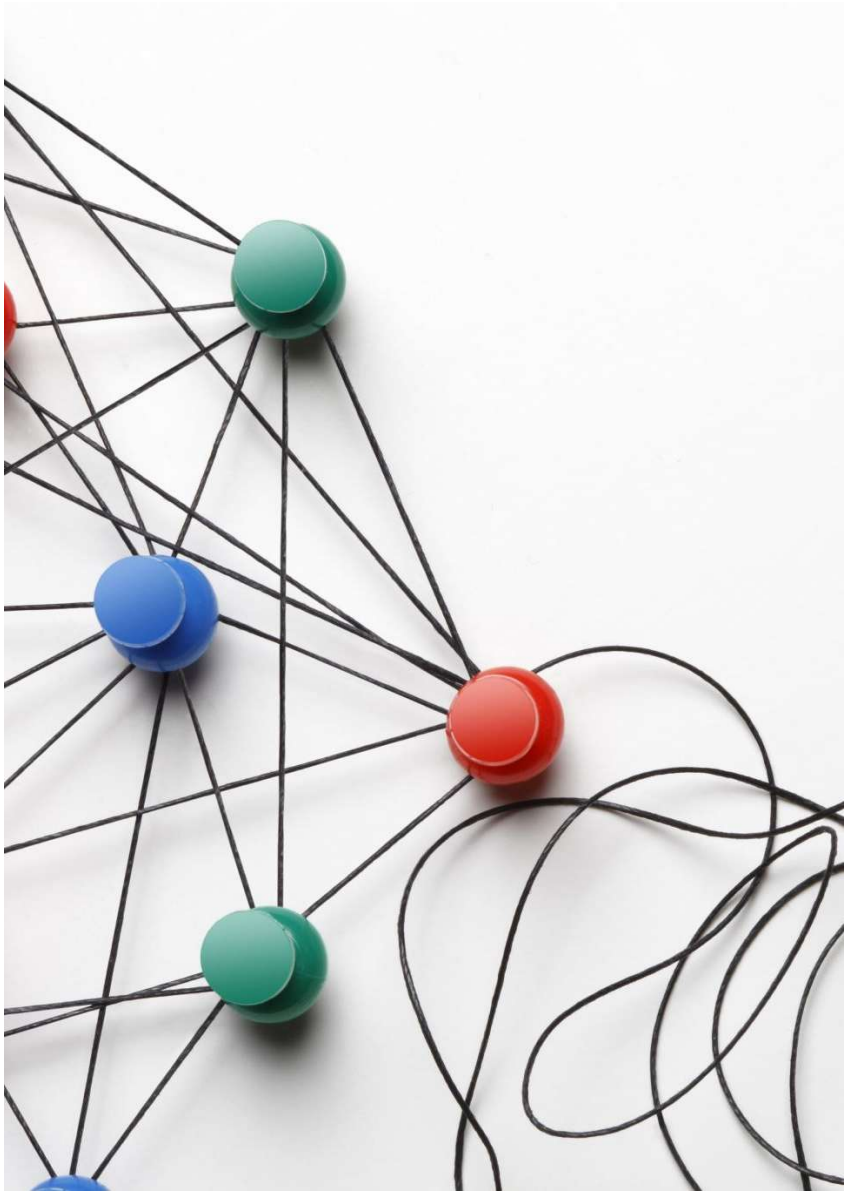
Virtual environments allow programmers to define what is allowed to support an application

Multiple virtual environments can exist on the same computer, with one being active to provide context to a single application

This prevents cross-contamination of application dependencies through isolation of those dependencies.

Objectives

- By the end of this session, you will be able to:
  - Explain what a Python virtual environment is and why it is needed.
  - Create and activate a virtual environment using Python's venv module.
  - Create and activate a virtual environment using uv.
  - Create and manage environments using Conda.
  - Install packages safely within an isolated environment.



Virtual Python Environments

More complicated Python applications include dependencies which are not included with a standard Python installation.

There is a public repository of Python packages which can be used to include functionality others have developed:

- pypi.org – Default package repository available by installing Python
- conda-forge.org – Package repository available with Anaconda

When developing multiple applications, importing dependent packages needed for both projects may result in conflicts.

Extraneous packages can consume resources unnecessarily and may interfere with the application desired behavior.

Specifying dependencies allow for more consistent installation of applications by ensuring all dependencies are satisfied

Tooling for Using Python Virtual Environments

venv

- Installs with Python
- Very light weight
- Minimal features

uv

- Additional installation
- More extensive set of functions than venv which ships with Python
- Tends to run faster than the native components

conda

- Additional installation
- Very heavy-weight installation
- Much more flexible and more functionality than venv
- Tends to be slow especially for more complex development environments

Installing package - pip

- pip is a python package installer
  - The name “pip” means “pip installs packages”
- pip can be used to install packages from pypi.org which is the “Python Package Index”.
- Examples:
  - pip install numpy
  - pip install scipy>=1.16
  - pip install scipy<1.16
- In each case the specified package and all its dependencies are installed in the current environment

Command	Description
pip install numpy	Install the lates version of the numpy package into the current environment
pip install scipy>=1.16	Install version 1.16 or later of the scipy package.
pip install scipy<1.16	Install the latest version of the scipy earlier than version 1.16

Using pip to manage application dependencies

- We can create a file listing all the dependencies of an application
- We can then use pip to install those elements and their collective dependencies
- To invoke, do the following (NOTE: the sample at right is meant to be illustrative only and is not intended to be used… see sample in GitHub repo):

```
pip install –r requirements.txt
```

- requirements.txt

```
requests==2.24.0
Flask==1.1.2
numpy>=1.18.5,<2.0
pandas>=1.0.3,!=1.1.0,<2.0
matplotlib>=3.2.2,<4.0
six!=1.16.0
virtualenv~=20.15
pycrumbs
```

venv

venv is included with python when you install it

It is used to create local environments that contain project dependencies

Allows isolation of those dependencies from larger environment and other projects

Add packages to an environment using pip after activating the environment

Creates a folder containing all of the elements associated with the virtual environment

See venv documentation and pip documentation for additional information

Using venv

Operation	Windows	Linux / Mac
Create new environment	`python -m venv myenv`	`python -m venv myenv`
Activating environment	`myenv\Scripts\activate`	`source myenv/bin/activate`
Deactivating environment	`deactivate`	`deactivate`
Installing packages	`pip install package`	`pip install package`
Remove package	`pip uninstall package`	`pip uninstall package`
Remove an environment	`rmdir /s /q myenv`	`rm -rf myenv`

uv

- uv can create and manage virtual environments
- uv is a drop-in replacement for pip + venv
- uv is a separate tool which performs functions like those in the Python distribution
- uv must be installed separate from the core python tooling
- uv is written in Rust and operates much faster than the native Python tools
- See uv documentation for additional details

Using uv

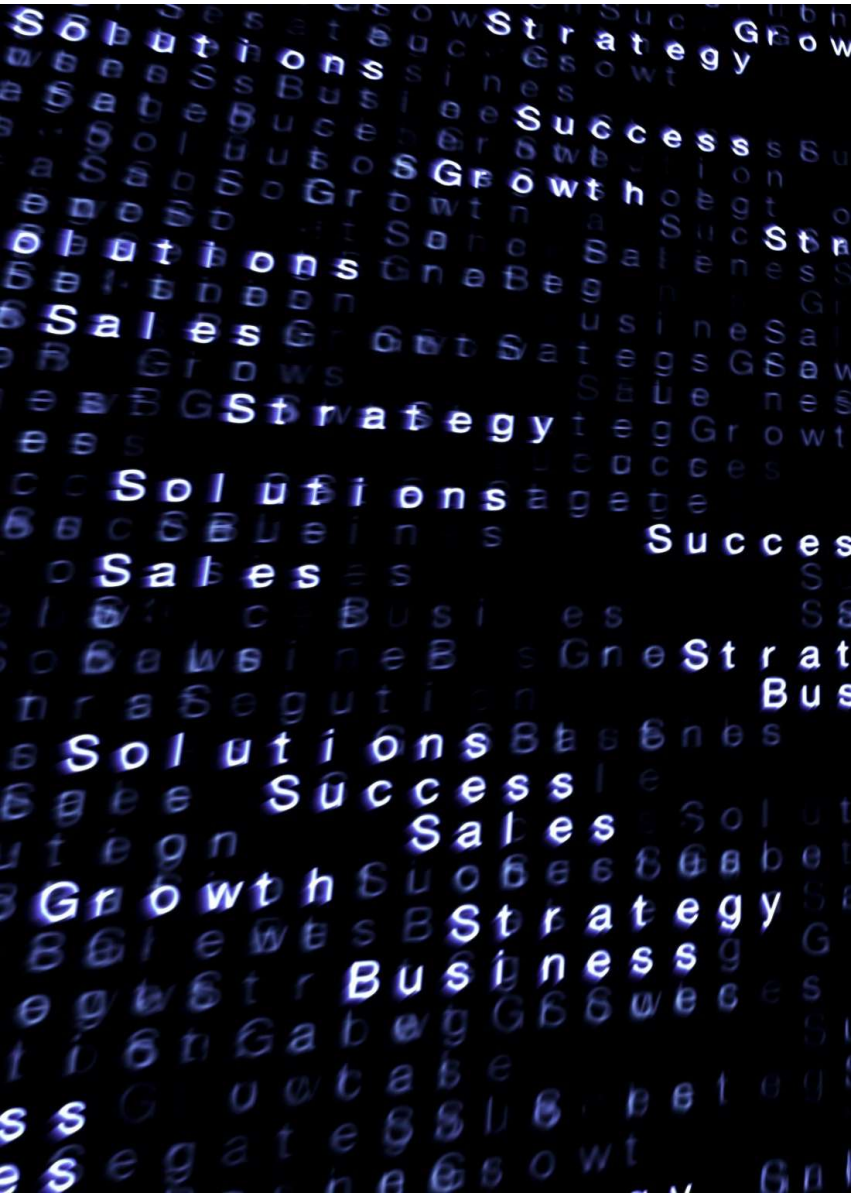
Operation	Windows	Linux / Mac
Installing uv	`pip install uv`	`pip install uv`
Create new environment	`uv venv` *`myenv`*	`uv venv` *`myenv`*
Activating environment	*`myenv`*`\Scripts\activate`	`source` *`myenv`*`/bin/activate`
Deactivating environment	`Deactivate`	`deactivate`
Installing packages	`uv pip install` *`package`*	`uv pip install` *`package`*
Listing all installed packaged	`uv pip freeze`	`uv pip freeze`
Remove package	`uv pip uninstall` *`package`*	`uv pip uninstall` *`package`*
Remove an environment	`rmdir /s /q` *`myenv`*	`rm –rf` *`myenv`*

anaconda

- Anaconda is a heavy weight package and virtual environment management system (conda documentation)
- It is widely used to support scientific computing, artificial intelligence / machine learning and larger package management for larger scale projects
- There is a GUI based tool, Anaconda, that is pretty, but not as flexible or useful as the command line tools
- The miniconda package includes all of the command line tools and none of the GUI tools.
- An open-source distribution of miniconda is conda-forge (documentation).
- Much of the conda functionality is available in another tool available with conda-forge, called mamba, which Is faster than using conda
- Environments are folders in the user <home-dir>/.conda/envs

Using anaconda (miniforge)

Operation	Windows	Linux / Mac
Installing miniforge	Download and run the Windows executable	Download and run the linux or macOS arm64 or macOS intel executable
Create new environment	`conda create -n myenv` `conda env create -f env.yaml`	`conda create -n myenv` `conda env create -f env.yaml`
Activating environment	`conda activate myenv`	`conda activate myenv`
Deactivating environment	`conda deactivate`	`conda deactivate`
Installing packages	`conda install package`	`conda install package`
Listing all installed packaged	`conda list`	`conda list`
List all environments	`conda env list`	`conda env list`
Remove package	`conda uninstall package`	`conda uninstall package`
Remove an environment	`conda env remove -n myenv`	`conda env remove -n myenv`



Activities

- Create a venv based virtual environment satisfying all of the dependencies in 21_environments/requirements.yaml
- Use uv to create an equivalent virtual environment and note the time difference.
- Create a conda environment using 21_environments/myenv.yaml
- Delete the just made conda environment and use “mamba” to recreate it instead and note the time difference